

Install Kubernetes Cluster on Ubuntu 20.04 using K3s

By [Victor Shamallah](#) - November 23, 2021

Kubernetes is a container orchestration tool for hybrid cloud platforms. It was designed by Google. Kubernetes allows users to manage docker containers across different cloud and virtualization platforms. K3s is a lightweight kubernetes deployment binary that is meant to reduce the 'heavily loaded' k8s deployment. K3s is being developed by [Rancher](#).

In this post we shall cover how to install a Kubernetes cluster on Ubuntu 20.04 LTS using k3s. You can check out how to install k8s on [Ubuntu 20.04 with kubeadm](#) for an alternative.

Kubernetes architecture involves a Master node and Worker Nodes. Their functions are as follows:

- **Master** – controls the cluster, API calls, e.t.c.
- **Workers** – these handles the workloads, where the pods are deployed and applications ran. They can be added and removed from the cluster.

So to setup a k3s cluster you need at least two hosts, the *master* node and one *worker* node. In this post we shall be using one master node and two worker nodes.

We need to prepare our hosts to be able to run k3s in a cluster. Use the following steps to install k3s cluster on Ubuntu 20.04:

Step 1: Update Ubuntu system

With your servers installed with Ubuntu 20.04, update and upgrade them:

```
sudo apt update
sudo apt -y upgrade && sudo systemctl reboot
```

Step 2: Map the hostnames on each node

Make sure you have the hostnames mapped on each node. This is by adding the IP and hostname of each node in the `/etc/hosts` file of each host.

In our setup, it is as follows:

```
$ sudo vim /etc/hosts
172.16.10.3 master
```

```
172.16.10.4 worker01
172.16.10.10 worker02
```

This has to be done on all the hosts for you to use DNS names.

Step 3: Install Docker on Ubuntu 20.04

The next step is to install docker on on the hosts. As discussed before, Kubernetes is used to manage Docker containers on hybrid cloud infrastructure. Thus we need to have docker up and running on all the nodes before we can setup K3s.

Add Docker APT repository:

```
sudo apt update
sudo apt install apt-transport-https ca-certificates curl
software-properties-common -y
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo
apt-key add -
sudo add-apt-repository "deb [arch=amd64]
https://download.docker.com/linux/ubuntu focal stable"
```

Install Docker CE on Ubuntu 20.04:

```
sudo apt update
sudo apt install docker-ce -y
```

This has to be done on all the hosts including the master node. After successful installation, start and enable the service.

```
sudo systemctl start docker
sudo systemctl enable docker
```

You can also check if the service is started and is running:

```
$ systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service;
   enabled; vendor preset: enabled)
   Active: active (running) since Fri 2020-10-30 08:30:39
   CET; 19s ago
   TriggeredBy: ● docker.socket
   Docs: https://docs.docker.com
   Main PID: 9275 (dockerd)
```

Tasks: 8

Memory: 35.6M

CGroup: /system.slice/docker.service

└─9275 /usr/bin/dockerd -H fd:// --

containerd=/run/containerd/containerd.sock

```
Oct 30 08:30:39 ubuntu dockerd[9275]: time="2020-10-30T08:30:39.460569874+01:00" level=warning msg="Your kernel does not support cgroup rt runtime"
Oct 30 08:30:39 ubuntu dockerd[9275]: time="2020-10-30T08:30:39.460672730+01:00" level=warning msg="Your kernel does not support cgroup blkio weight"
Oct 30 08:30:39 ubuntu dockerd[9275]: time="2020-10-30T08:30:39.460820506+01:00" level=warning msg="Your kernel does not support cgroup blkio weight_device"
Oct 30 08:30:39 ubuntu dockerd[9275]: time="2020-10-30T08:30:39.461286028+01:00" level=info msg="Loading containers: start."
Oct 30 08:30:39 ubuntu dockerd[9275]: time="2020-10-30T08:30:39.620849970+01:00" level=info msg="Default bridge (docker0) is assigned with an IP address 172.17.0>
Oct 30 08:30:39 ubuntu dockerd[9275]: time="2020-10-30T08:30:39.717281156+01:00" level=info msg="Loading containers: done."
Oct 30 08:30:39 ubuntu dockerd[9275]: time="2020-10-30T08:30:39.754253372+01:00" level=info msg="Docker daemon"
commit=4484c46d9d graphdriver(s)=overlay2 version>
Oct 30 08:30:39 ubuntu dockerd[9275]: time="2020-10-30T08:30:39.755461813+01:00" level=info msg="Daemon has completed initialization"
Oct 30 08:30:39 ubuntu systemd[1]: Started Docker Application Container Engine.
```

Add your user to Docker group to avoid typing sudo everytime you run docker commands.

```
sudo usermod -aG docker ${USER}
newgrp docker
```

Step 4: Setup the Master k3s Node

In this step, we shall install and prepare the master node. This involves installing the k3s service and starting it.

```
curl -sfL https://get.k3s.io | sh -s - --docker
```

Run the command above to install k3s on the master node. The script installs k3s and starts it automatically.

```
[INFO] Finding release for channel stable
[INFO] Using v1.18.9+k3s1 as release
[INFO] Downloading hash
https://github.com/rancher/k3s/releases/download/v1.18.9+k3s1/s
ha256sum-amd64.txt
[INFO] Downloading binary
https://github.com/rancher/k3s/releases/download/v1.18.9+k3s1/k
3s
[INFO] Verifying binary download
[INFO] Installing k3s to /usr/local/bin/k3s
[INFO] Creating /usr/local/bin/kubectl symlink to k3s
[INFO] Creating /usr/local/bin/crictl symlink to k3s
[INFO] Skipping /usr/local/bin/ctr symlink to k3s, command
exists in PATH at /usr/bin/ctr
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-
uninstall.sh
[INFO] env: Creating environment file
/etc/systemd/system/k3s.service.env
[INFO] systemd: Creating service file
/etc/systemd/system/k3s.service
[INFO] systemd: Enabling k3s unit
Created symlink /etc/systemd/system/multi-
user.target.wants/k3s.service →
/etc/systemd/system/k3s.service.
[INFO] systemd: Starting k3s
```

To check if the service installed successfully, you can use:

```
$ systemctl status k3s
● k3s.service - Lightweight Kubernetes
   Loaded: loaded (/etc/systemd/system/k3s.service; enabled;
 vendor preset: enabled)
   Active: active (running) since Fri 2020-10-30 08:32:46
```

CET; 53s ago

Docs: <https://k3s.io>

Process: 11151 ExecStartPre=/sbin/modprobe br_netfilter

(code=exited, status=0/SUCCESS)

Process: 11152 ExecStartPre=/sbin/modprobe overlay

(code=exited, status=0/SUCCESS)

Main PID: 11153 (k3s-server)

Tasks: 0

Memory: 420.9M

CGroup: /system.slice/k3s.service

└─11153 /usr/local/bin/k3s server --docker

Oct 30 08:33:32 ubuntu k3s[11153]: I1030 08:33:32.532935

11153 reconciler.go:196] operationExecutor.UnmountVolume
started for volume "helm-traefik-token-dmfz6">

Oct 30 08:33:32 ubuntu k3s[11153]: I1030 08:33:32.533481

11153 reconciler.go:196] operationExecutor.UnmountVolume
started for volume "values" (UniqueName: "kub>

Oct 30 08:33:32 ubuntu k3s[11153]: W1030 08:33:32.533849

11153 empty_dir.go:453] Warning: Failed to clear quota on
/var/lib/kubelet/pods/d1fae0b1-f3ac-481a-b6a>

Oct 30 08:33:32 ubuntu k3s[11153]: I1030 08:33:32.538649

11153 operation_generator.go:782] UnmountVolume.TearDown
succeeded for volume "kubernetes.io/configmap>

Oct 30 08:33:32 ubuntu k3s[11153]: I1030 08:33:32.555370

11153 operation_generator.go:782] UnmountVolume.TearDown
succeeded for volume "kubernetes.io/secret/dl>

Oct 30 08:33:32 ubuntu k3s[11153]: I1030 08:33:32.653113

11153 reconciler.go:319] Volume detached for volume "helm-
traefik-token-dmfz6" (UniqueName: "kubernete>

Oct 30 08:33:32 ubuntu k3s[11153]: I1030 08:33:32.653137

11153 reconciler.go:319] Volume detached for volume "values"
(UniqueName: "kubernetes.io/configmap/dlf>

Oct 30 08:33:33 ubuntu k3s[11153]: W1030 08:33:33.978774

11153 pod_container_deletor.go:77] Container
"23d34e0acde3f4ab7e41d99e9cff16e7ba8cd76122158dfc18b64ae0>

Oct 30 08:33:33 ubuntu k3s[11153]: W1030 08:33:33.997821

11153 pod_container_deletor.go:77] Container
"a52fe91aa3e7b137ebe4dd993e358c382f96b5df342cb1c098acadb6>

Oct 30 08:33:35 ubuntu k3s[11153]: W1030 08:33:35.022454

11153 pod_container_deletor.go:77] Container
"2b9f302427059276bc2e1d012db5cbcaeb72ed373cb52218edb6eb4b

You can check if the master node is working by :

```
sudo kubectl get nodes -o wide
```

The output should be something like this:

```
root@mastre:~# sudo kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP
master	Ready	master	3m2s	v1.18.9+k3s1	172.16.10.3

```

EXTERNAL-IP    OS-IMAGE          KERNEL-VERSION    CONTAINER-RUNTIME
<none>         Ubuntu 20.04.1 LTS  5.4.0-52-generic  docker://19.3.8

```

Step 5: Allow ports on firewall

We need to allow ports that will be used to communicate between the master and the worker nodes. The ports are 443 and 6443.

```
sudo ufw allow 6443/tcp
sudo ufw allow 443/tcp
```

You need to extract a token from the master that will be used to join the nodes to the master.

On the master node:

```
sudo cat /var/lib/rancher/k3s/server/node-token
```

You will then obtain a token that looks like:

```
K1078f2861628c95aa328595484e77f831adc3b58041e9ba9a8b2373926c8b0
34a3::server:417a7c6f46330b601954d0aaaa1d0f5b
```

Step 6: Install k3s on worker nodes and connect them to the master

The next step is to install k3s on the worker nodes. Run the commands below to install k3s on worker nodes:

```
curl -sL http://get.k3s.io | K3S_URL=https://<master_IP>:6443
K3S_TOKEN=<join_token> sh -s - --docker
```

Where `master_IP` is the IP of the master node and `join_token` is the token obtained from the master. e.g:

```
curl -sfL http://get.k3s.io | K3S_URL=https://172.16.10.3:6443
K3S_TOKEN=K1078f2861628c95aa328595484e77f831adc3b58041e9ba9a8b2
373926c8b034a3::server:417a7c6f46330b601954d0aaaa1d0f5b sh -s -
--docker
```

You can verify if the k3s-agent on the worker nodes is running by:

```
sudo systemctl status k3s-agent
```

To verify that our nodes have successfully been added to the cluster, run :

```
sudo kubectl get nodes
```

Your output should look like:

```
root@master:~# kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP
EXTERNAL-IP	OS-IMAGE			KERNEL-VERSION	
CONTAINER-RUNTIME					
master	Ready	master	14m	v1.18.9+k3s1	172.16.10.3
<none>	Ubuntu 20.04.1 LTS			5.4.0-52-generic	
docker://19.3.8					
worker02	Ready	<none>	90s	v1.18.9+k3s1	172.16.10.10
<none>	Ubuntu 20.04.1 LTS			5.4.0-52-generic	
docker://19.3.8					
worker01	Ready	<none>	41s	v1.18.9+k3s1	172.16.10.4
<none>	Ubuntu 18.04.5 LTS			4.15.0-122-generic	
docker://19.3.6					

This shows that we have successfully setup our k3s cluster ready to deploy applications to it.

Step 7: Deploy Addons to K3s

K3s is a lightweight kubernetes tool that doesn't come packaged with all the tools but you can install them separately.

Install Helm Commandline tool on k3s

- Download the latest version of Helm commandline tool from this [page](#).
- Extract the tar file using `tar -xvzf <downloaded-file>`
- Move the binary file to `/usr/local/bin/helm`

```
sudo mv linux-amd64/helm /usr/local/bin/helm
```

Check version

```
$ helm version
```

Add the helm chart repository to allow installation of applications using helm:

```
helm repo add stable https://charts.helm.sh/stable
helm repo update
```

Step 8: Deploy an application on k3s

We can now deploy a test application on the K3s cluster.

Deploy Nginx Web-proxy on K3s

Nginx can be used as a web proxy to expose ingress web traffic routes in and out of the cluster.

We can install nginx web-proxy using Helm:

```
helm install nginx-ingress stable/nginx-ingress --namespace
kube-system \
--set defaultBackend.enabled=false
```

We can test if the application has been installed by:

```
$ kubectl get pods -n kube-system -l app=nginx-ingress -o wide
```

The output should be like:

```
root@master:~# kubectl get pods -n kube-system -l app=nginx-
ingress -o wide
```

NAME	READY	STATUS
RESTARTS	AGE	IP
NODE	NOMINATED	NODE
READINESS	GATES	
nginx-ingress-controller-56547fb57-vsjudm	1/1	Running
28s	10.42.2.6	worker01
<none>	<none>	

We have successfully deployed nginx web-proxy on k3s.

Step 9: Removing k3s (Bonus)

To remove k3s on the worker nodes, execute:

```
sudo /usr/local/bin/k3s-agent-uninstall.sh  
sudo rm -rf /var/lib/rancher
```

To remove k3s on the master node, execute:

```
sudo /usr/local/bin/k3s-uninstall.sh  
sudo rm -rf /var/lib/rancher
```

Conclusion

In this article, we have been able to deploy k3s on a self hosted environment. We have also deployed helm and nginx on our k3s cluster.

You can find more articles on Kubernetes below:

- [Secure Access to Linux Systems and Kubernetes With Teleport](#)
- [Install Kubernetes Cluster on Ubuntu 20.04 with kubeadm](#)
- [How To Install MicroK8s Kubernetes Cluster on CentOS 8](#)

**Your support is our everlasting motivation,
that cup of coffee is what keeps us going!**

As we continue to grow, we would wish to reach and impact more people who visit and take advantage of the guides we have on our blog. This is a big task for us and we are so far extremely grateful for the kind people who have shown amazing support for our work over the time we have been online.

Thank You for your support as we work to give you the best of guides and articles. Click below to buy us a coffee.

Victor Shamallah

A systems engineer with vast skills in cloud computing, virtualization, systems administration, automated orchestration, containers, systems deployment, etc.

